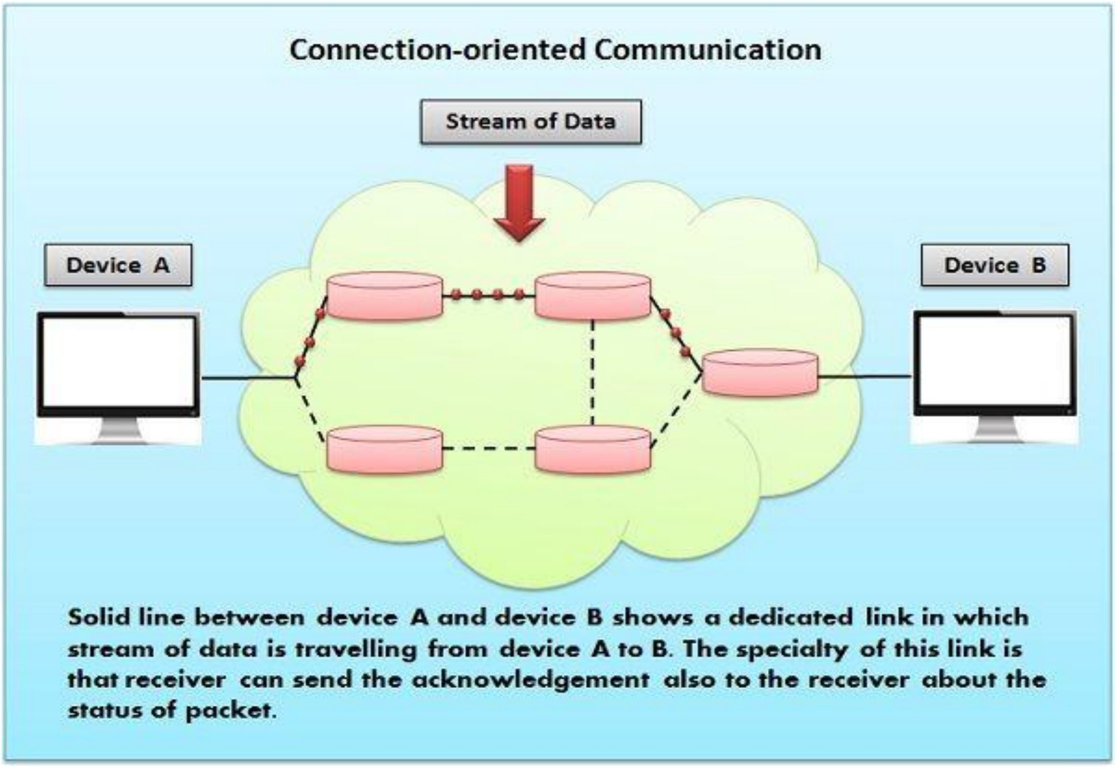
Connection-Oriented Service

- A connection-oriented service **establishes a dedicated communication path between the sender and receiver before any data is transferred.**
- This process **involves a handshake** to set up the connection, ensuring both parties are ready for communication.



- Once the connection is established, data packets are transmitted sequentially and reliably.
- The connection remains active until all data has been successfully transferred. Afterward, the connection is terminated.
- A common example of a connection-oriented service is TCP (Transmission Control Protocol), which guarantees error-free and in-order delivery of packets.

Examples:

- **TCP (Transmission Control Protocol):** used for reliable data transfer in applications like HTTP, FTP and email.

- **Telephone calls:** where a dedicated communication channel is established between two users.

Key Features:

- **Dedicated Connection:** A logical/physical link is established before data transfer.
- **Reliable Transmission:** Error detection, acknowledgment and retransmission mechanisms ensure reliability.
- **Sequencing:** Packets arrive in the correct order.
- **High Overhead:** Extra resources are used to establish and maintain the connection.

Advantages:

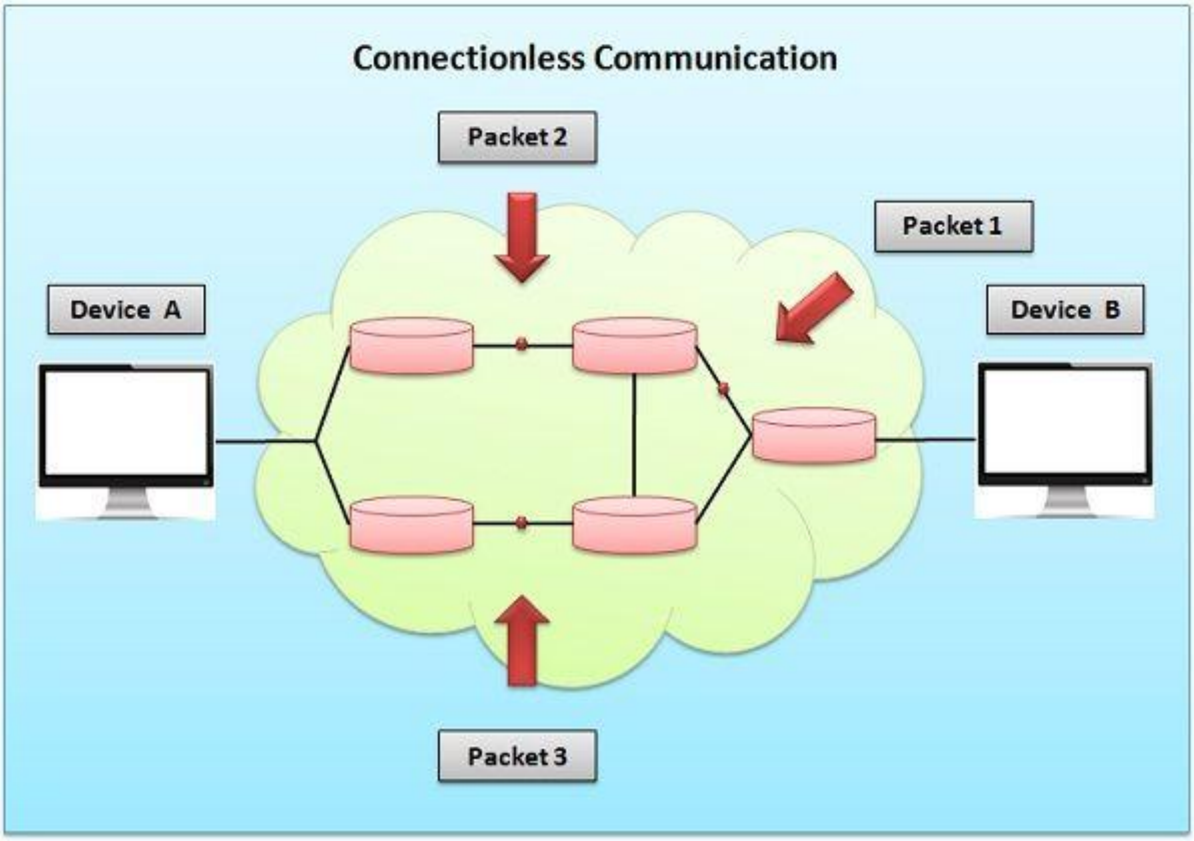
- Reliable and error-free data delivery
- Guaranteed data sequencing
- Suitable for large, continuous data transfers
- Ensures retransmission of lost data

Disadvantages:

- Higher latency due to connection setup
- Greater resource consumption
- Less efficient for small or one-time messages

Connectionless Service

- A connectionless service **transmits data packets without first establishing a dedicated path between sender and receiver.**
- Each packet (often called a datagram) is treated independently and may follow different routes to reach the destination.



- There is no guarantee of delivery order, reliability or error correction, but the transmission is faster and more scalable.
- The most common example is UDP (User Datagram Protocol), used in real-time applications where speed is more important than reliability.

Examples:

- **UDP (User Datagram Protocol):** used for DNS queries, streaming and gaming.
- **Postal services analogy:** letters are sent independently, without confirmation of delivery.

Key Features:

- **No Connection Setup:** Data is sent immediately.
- **Independent Packets:** Each packet travels separately.
- **Faster Transmission:** No overhead for connection management.
- **Unreliable:** Packets may be lost or arrive out of order.

Advantages:

- Low latency and faster communication
- Efficient for small or time-sensitive data transfers
- Scalable for large networks with multiple users
- Simple implementation and less overhead

Disadvantages:

- No error control or acknowledgment
- Packets can be lost, duplicated or reordered
- Not suitable for large or critical data transmissions

Difference Between Connection-Oriented and Connectionless Services

Parameter	Connection-Oriented Service	Connectionless Service
Connection Setup	Requires a connection before data transfer	No connection setup needed
Reliability	Reliable; ensures delivery, error detection and retransmission	Unreliable; no delivery or error guarantee
Data Sequencing	Packets delivered in correct order	Packets may arrive out of order
Overhead	High due to connection management	Low; no connection maintenance
Speed	Slower due to setup and acknowledgments	Faster as data is sent directly
Resource Usage	Consumes more resources (buffers, control info)	Minimal resource usage
Best Suited For	File transfer, web browsing, emails (TCP)	Streaming, DNS, VoIP (UDP)

Parameter	Connection-Oriented Service	Connectionless Service
Example Protocol	TCP (Transmission Control Protocol)	UDP (User Datagram Protocol)